



PYTHON

Test unitarios

1

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

Test unitarios

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios
 - Frameworks de pruebas incluidos en Python:
 - unittest
 - <https://docs.python.org/3.9/library/unittest.html>
 - <https://docs.python.org/3.9/library/unittest.html#classes-and-functions>
 - doctest
 - <https://docs.python.org/3/library/doctest.html>

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

unittest

<https://docs.python.org/3/library/unittest.html>

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Módulo unittest
- La clase que implementa los test debe heredar de unittest.TestCase
- Métodos: los métodos de test comienzan por test_

```
@classmethod
def setUpClass(cls) -> None:
    print("Se ejecuta antes de iniciar el proceso de test")

@classmethod
def tearDownClass(cls) -> None:
    print("Se ejecuta después de iniciar el proceso de test")

def setUp(self) -> None:
    print("Se ejecuta antes de ejecutar cada test")

def tearDown(self) -> None:
    print("Se ejecuta después de ejecutar cada test")

def test_prueba(self):
    self.assertEqual(3,4)
```

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios
 - Métodos de validación más usados.

Method	Checks that	New in
<code>assertEqual(a, b)</code>	<code>a == b</code>	
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	
<code>assertTrue(x)</code>	<code>bool(x) is True</code>	
<code>assertFalse(x)</code>	<code>bool(x) is False</code>	
<code>assertIs(a, b)</code>	<code>a is b</code>	3.1
<code>assertIsNot(a, b)</code>	<code>a is not b</code>	3.1
<code>assertIsNone(x)</code>	<code>x is None</code>	3.1
<code>assertIsNotNone(x)</code>	<code>x is not None</code>	3.1
<code>assertIn(a, b)</code>	<code>a in b</code>	3.1
<code>assertNotIn(a, b)</code>	<code>a not in b</code>	3.1
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>	3.2
<code>assertNotIsInstance(a, b)</code>	<code>not isinstance(a, b)</code>	3.2

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios

- Otros métodos:

- unittest.main() → Ejecuta los test de la clase
 - assertRaise → Comprueba que lanza una excepción.
 - fail → Provoca un error forzado.
 - skipTest → Hace que se salte el test.
 - self.skipTest("No aplica")

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios
 - unittest. Ejemplo: código a probar. calculadora.py

```
def sumar(s1, s2):  
    return s1+s2
```

```
def restar(r1, r2):  
    return r1*r2
```

```
def dividir(d1, d2):  
    return d1/d2
```


MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios

- Unittest. Ejemplo: test unitario.

```
import unittest
import calculadora

class TestCalculadora(unittest.TestCase):
    def test_suma(self):
        self.assertEqual(calculadora.sumar(10.3, 4.1), 14.4)

    def test_restar(self):
        self.assertEqual(calculadora.restar(8,2), 6)

    def test_dividir(self):
        self.assertEqual(calculadora.dividir(8,0), 4)

if __name__ == "__main__":
    unittest.main()
```

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

doctest

<https://docs.python.org/3/library/doctest.html>

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios
 - Doctest: incrustados en los comentarios

```
def sumar(s1, s2):  
    """  
    Suma dos números y devuelve el resultado  
    >>> sumar(3,4)  
    15  
    """  
    return s1+s2
```

MANEJO DE EXCEPCIONES Y CONTROL DE ERRORES

- Test unitarios
 - Doctest: ejecución desde la ejecución.
 - `python -m doctest -v calculadora.py`
 - Doctest: ejecución desde el código. Sin el parámetro `verbose=True` sólo muestra los errores por la consola.

```
import doctest
doctest.testmod(verbose=True)
```

```
> python calculadora.py
```